

Lab: Digital Certificate Using OpenSSL

Estimated time: 45 minutes

Overview

In this lab, you will learn how to create and manage digital certificates using OpenSSL. You will generate private keys, create certificate signing requests (CSRs), self-sign certificates, and verify certificates using OpenSSL commands.

Learning objectives:

- Generate private keys and CSRs
 - Create self-signed digital certificates
 - Verify digital certificates' authenticity
-

Prerequisites (Optional)

- Install OpenSSL on your system.
 - Basic knowledge of certificates and their use in secure communication is advantageous.
-

Initializing the Lab Environment

Follow the below steps to initialize the lab environment.

- Select **Terminal** and then select **New Terminal**. This will open a new terminal.

FileEditSelectionViewGoRunTerminalHelp

←→

☐

SKILLS ...

> DATABASES

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Appli...

Welcome X

Cloud IDE

Version 1.7.4

VS Code API Versio

What is Cloud

Cloud IDE is a full

Environment (IDE

debug your code

Getting Started

On your left,you'll find the instructions for the tasks you need to complete in this lab. Follow them step-by-step to achieve the objectives of this lab. You can also click the chat button to ask for help from Tai, your AI Teaching Assistant at any time.

On your right (here),is the Cloud IDE tool, a fully-featured Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.

Here are some key components of the Cloud IDE tool:

- **File Explorer:** Navigate through your files and directories. [Click HERE to open the File Explorer.](#)
- **Terminal:** Run your code and see the output here. [Click HERE to open a New Terminal.](#)
- **Skills Network Toolbox:** Access various technologies like databases and Embeddable AI with just a click. [Click HERE to toggle the Skills Network Toolbox.](#)

☒ Show welcome page on startup

New TerminalCtrl+Shift+`

New Terminal (With Profile)...

Choose Default Profile...

Run Task...

Run Build Task

Run Test Task

Rerun Last TaskCtrl+Shift+K

Show Running Tasks...

Restart Running Task...

Terminate Task...

Attach Task...

Configure Tasks...



SkillsNetwork

Open Workspace

Recent

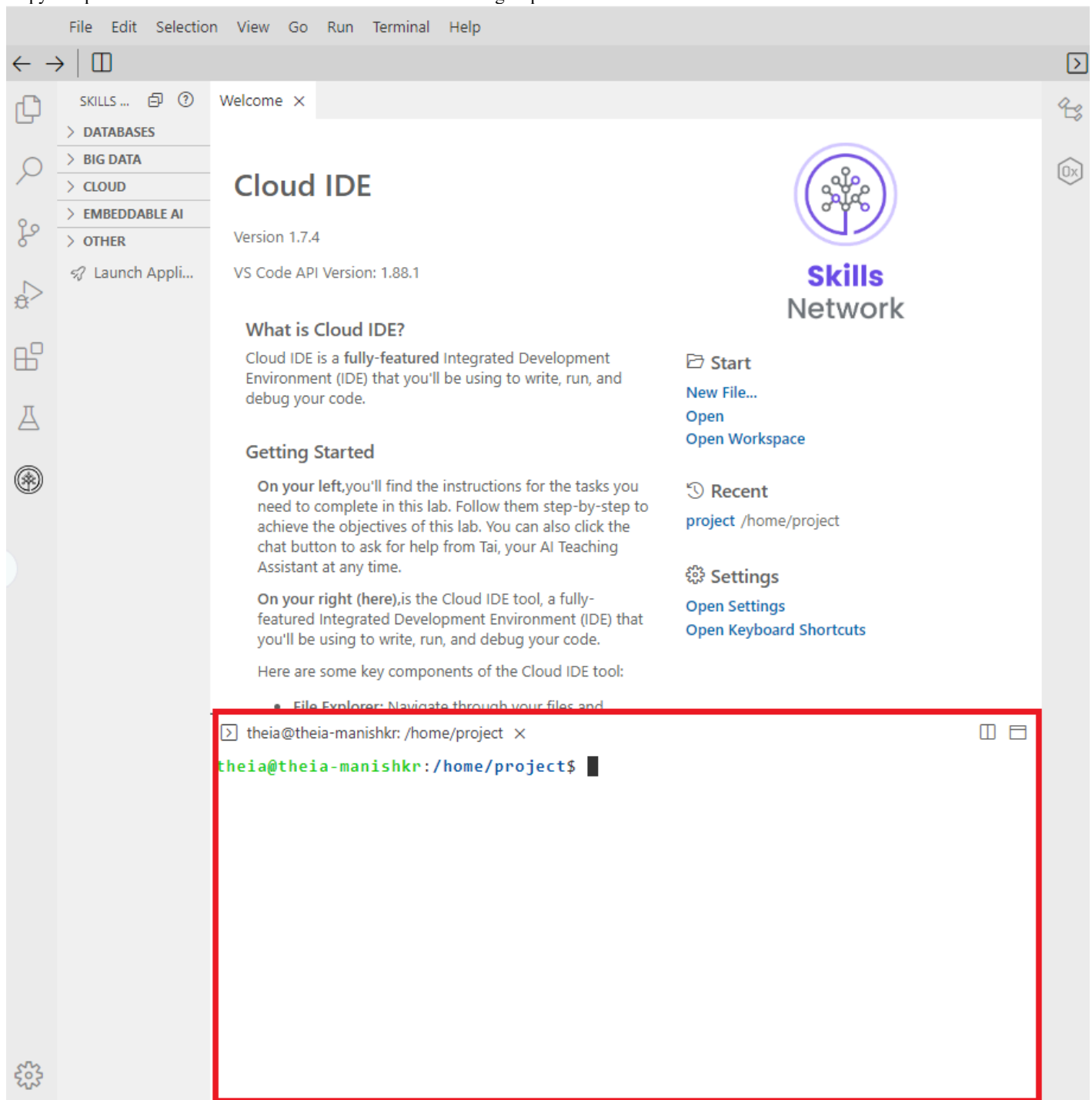
project /home/project

Settings

Open Settings

Open Keyboard Shortcuts

- Copy and paste the commands into the terminal for the remaining steps.



Step 1: Generate a Private Key

Run the below command to create a private key for the certificate:

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

Command breakdown:

- **genpkey**: Generates a private key.
 - **-algorithm RSA**: Specifies RSA as the key generation algorithm.
 - **-out private_key.pem**: Saves the private key in a file named `private_key.pem`.
 - **-pkeyopt rsa_keygen_bits:2048**: Generates a 2048-bit key for strong encryption.
-

Step 2: Generate a Certificate Signing Request (CSR)

Run the below command to generate a CSR using the private key:

```
openssl req -new -key private_key.pem -out certificate.csr
```

Command breakdown:

- **req -new**: Generates a new CSR.
- **-key private_key.pem**: Uses the private key to generate the CSR.
- **-out certificate.csr**: Saves the CSR to `certificate.csr`.

Prompt:

You will be asked to provide details such as:

- **Country Name (2 letter code)**: For example, US
 - **State or Province Name**: For example, California
 - **Locality Name**: For example, San Francisco
 - **Organization Name**: For example, Example Corp
 - **Organizational Unit Name**: For example, IT
 - **Common Name**: For example, `www.example.com`
 - **Email Address**: For example, admin@example.com
-

Step 3: Generate a Self-Signed Certificate

Run the below command to use the CSR for generating a self-signed certificate:

```
openssl x509 -req -in certificate.csr -signkey private_key.pem -out certificate.crt -days 365
```

Command breakdown:

- **x509 -req**: Specifies that the input is a CSR.
- **-in certificate.csr**: Input CSR file.
- **-signkey private_key.pem**: Uses the private key to sign the certificate.
- **-out certificate.crt**: Saves the self-signed certificate to `certificate.crt`.
- **-days 365**: Sets the validity period of the certificate to 365 days.

Step 4: Verify the Certificate

Run the below command to check the contents of the certificate:

```
openssl x509 -in certificate.crt -text -noout
```

Command breakdown:

- **x509 -in certificate.crt**: Specifies the certificate to inspect.
- **-text**: Displays the certificate in a human-readable format.
- **-noout**: Suppresses output of the raw certificate.

Step 5: Verify the Certificate Chain

Although this step is more relevant for CA-signed certificates, you can simulate verification with the self-signed certificate:

```
openssl verify -CAfile certificate.crt certificate.crt
```

Command breakdown:

- **verify**: Verifies the certificate.
- **-CAfile certificate.crt**: Specifies the certificate as its own CA (self-signed).

Expected output:

```
certificate.crt: OK
```

Step 6: Simulate Expiry and Renewal

Follow the steps below to simulate the expiration and renewal of the certificate.

Step A: Modify the validity period

To simulate the expired certificate, use a shorter **-days** value, for example:

```
openssl x509 -req -in certificate.csr -signkey private_key.pem -out expired_certificate.crt -days 1
```

***Note:** Step A simulates the creation of an expired certificate by setting a very short validity period (1 day).*

Step B: Renew the Certificate

Renew the expired certificate by reissuing it with an updated `-days` value.

```
openssl x509 -req -in certificate.csr -signkey private_key.pem -out renewed_certificate.crt -days 365
```

***Note:** Step B simulates the renewal of the expired certificate by generating a new certificate with a longer validity period (1 year in this case).*

Step 7: Export the Certificate in Different Formats

Run the below command to export the certificate in different formats.

Step A: Convert to DER format

```
openssl x509 -in certificate.crt -outform DER -out certificate.der
```

Command breakdown:

- `-outform DER`: Specifies DER format (binary).
- `-out certificate.der`: Saves the output in `certificate.der`.

Step B: Convert to PKCS#12 format

Combine the private key and certificate into a PKCS#12 bundle:

```
openssl pkcs12 -export -inkey private_key.pem -in certificate.crt -out certificate.p12
```

Command breakdown:

- `pkcs12 -export`: Creates a PKCS#12 file.

- `-inkey private_key.pem`: Includes the private key.
 - `-in certificate.crt`: Includes the certificate.
 - `-out certificate.p12`: Saves the output in certificate.p12.
-

Exercises

Exercise 1: Certificate Revocation Using a Certificate Revocation List (CRL)

Objective: Simulate revoking a client certificate using a CRL and validate its status.

Steps hint:

Step 1: Install OpenSSL (if not already installed)

Ensure OpenSSL is installed on your system.

- **For Ubuntu/Debian:**

```
sudo apt update
sudo apt install openssl
```

- **For CentOS/RHEL:**

```
sudo yum install openssl
```

Step 2: Set up the certificate authority (CA)

1. Create necessary directories:

```
mkdir -p ca/{certs,crl,newcerts,private}
touch ca/index.txt
echo 1000 > ca/serial
```

2. Create the OpenSSL configuration file:

Create a file named `openssl.cnf` in the CA directory:

```
nano ca/openssl.cnf
```

Content of openssl.cnf:

```
[ ca ]
default_ca = CA_default
[ CA_default ]
dir = ./ca
database = $dir/index.txt
serial = $dir/serial
new_certs_dir = $dir/newcerts
certificate = $dir/certs/ca.crt
private_key = $dir/private/ca.key
default_md = sha256
default_days = 365
policy = policy_any
crlnumber = $dir/crlnumber
crl_dir = $dir/crl
default_crl_days = 30
crl_extensions = crl_ext
[ policy_any ]
commonName = supplied
stateOrProvinceName = optional
countryName = optional
[ crl_ext ]
authorityKeyIdentifier = keyid:always
```

Step 3: Generate the CA key and certificate

1. Generate the private key for the CA:

```
openssl genrsa -out ca/private/ca.key 2048
```

2. Generate the self-signed CA certificate:

```
openssl req -new -x509 -key ca/private/ca.key -out ca/certs/ca.crt -days 365 -subj "/CN=MyTestCA"
```

Step 4: Create and sign a client certificate

1. Generate the client's private key:

```
openssl genrsa -out client.key 2048
```

2. Generate the certificate signing request (CSR):

```
openssl req -new -key client.key -out client.csr -subj "/CN=ClientCertificate"
```

3. Sign the CSR with the CA:

```
openssl ca -config ca/openssl.cnf -in client.csr -out client.crt -batch
```

Step 5: Revoke the client certificate

1. Revoke the certificate:

```
openssl ca -config ca/openssl.cnf -revoke client.crt
```

Step 6: Generate the CRL file

1. Generate the CRL:

```
openssl ca -config ca/openssl.cnf -gencrl -out ca/crl/crl.pem
```

Step 7: Verify the revoked certificate

1. Verify the revoked certificate against the CRL:

```
openssl verify -crl_check -CAfile ca/certs/ca.crt -CRLfile ca/crl/crl.pem client.crt
```

- o You should see an error like:

```
client.crt: certificate revoked
```

Step 8: Validate and reflect

1. Inspect the CRL:

```
openssl crl -in ca/crl/crl.pem -text -noout
```

Steps to fix the error:

Step 1. Ensure the crlnumber file exists:

The crlnumber file is required by OpenSSL to track CRL versions. You need to create this file manually if it doesn't already exist.

1. Navigate to your CA directory:

```
cd ca
```

2. Create the `crlnumber` file and initialize it:

```
echo 1000 > crlnumber
```

- **Explanation:**

This initializes the CRL number with 1000, and OpenSSL will increment it for each new CRL generated.

Step 2. Check OpenSSL Configuration (`openssl.cnf`)

Make sure the `crlnumber` directive is correctly configured in your `openssl.cnf` file.

```
[ CA_default ]
dir = ./ca
database = $dir/index.txt
serial = $dir/serial
new_certs_dir = $dir/newcerts
certificate = $dir/certs/ca.crt
private_key = $dir/private/ca.key
default_crl_days = 30
crlnumber = $dir/crlnumber    # Ensure this line points to the correct crlnumber file
```

Step 3. Re-run the CRL generation command:

After creating and initializing the `crlnumber` file, try generating the CRL again:

```
openssl ca -config openssl.cnf -gencrl -out crl.pem
```

Step 4. Verify permissions

Ensure all necessary files (`index.txt`, `serial`, `crlnumber`) are readable and writable by your user.

```
chmod 644 ca/index.txt ca/serial ca/crlnumber
```

Step 5. Verify CRL generation

If the above steps succeed, verify the CRL:

```
openssl crl -in crl.pem -text -noout
```

Exercise 2: Extract and Inspect the Certificate Fields

Objective:

Analyze and extract specific fields (for example, issuer, validity) from a certificate file using OpenSSL.

Step 1: Set Up your environment

1. Ensure that OpenSSL is installed:

If OpenSSL isn't already installed, follow the below steps:

- **Ubuntu/Debian:**

```
sudo apt update
sudo apt install openssl
```

- **CentOS/RHEL:**

```
sudo yum install openssl
```

2. Place the certificate in your working directory:

Copy the certificate file (client.crt or your desired certificate) to your working directory.

Step 2: Extract and inspect the certificate fields

1. View complete certificate details:

```
openssl x509 -in client.crt -text -noout
```

2. Extract specific fields:

- **Issuer information:**

```
openssl x509 -in client.crt -noout -issuer
```

- **Subject information:**

```
openssl x509 -in client.crt -noout -subject
```

- **Validity period:**

```
openssl x509 -in client.crt -noout -dates
```

- **Certificate serial number:**

```
openssl x509 -in client.crt -noout -serial
```

- **Extensions (if present):**

```
openssl x509 -in client.crt -noout -text | grep -A10 "X509v3 extensions"
```

Step 3: Save the extracted information to files

1. Save issuer information to the file:

```
openssl x509 -in client.crt -noout -issuer > issuer_info.txt
```

2. Save subject information to the file:

```
openssl x509 -in client.crt -noout -subject > subject_info.txt
```

3. Save validity period to the file:

```
openssl x509 -in client.crt -noout -dates > validity_info.txt
```

Step 4: Verify the extracted information

1. Display saved files:

- **Issuer Information:**

```
cat issuer_info.txt
```

- **Subject information:**

```
cat subject_info.txt
```

- **Validity period:**

```
cat validity_info.txt
```

Conclusion

In this lab, you learned to generate private keys, create self-signed certificates, and how to verify them. These skills are essential for understanding Public Key Infrastructure (PKI) and secure communications. By practicing with OpenSSL, you gained hands-on experience with digital certificate creation and management.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.